

BSc (Hons) in **Computer Science****SE3112 : Advanced Software Engineering**

Year 3 · Semester 1 · 2026

Faculty of Computing

LAB SHEET

04

Lab 04: Graph Coverage, Part 3

Objective: This lab completes the graph coverage material from Lecture 02. It covers the formal vocabulary of paths and subpaths, Prime Path Coverage (the criterion designed specifically to make loops testable), and Data Flow Coverage (All-Defs, All-Uses, and All-DU-Paths Coverage). Students will build one CFG and use it for both the prime path work and the data flow work, the same way the lecture's own worked example does.

Introduction

Labs 02 and 03 covered building a CFG, and Node, Edge, and Edge-Pair Coverage. Lecture 02 covers more ground than that: the formal vocabulary used to talk about paths in a graph, a criterion built specifically to make looping code testable without demanding infinite test paths, and a whole second family of coverage criteria based on tracking variables rather than just control flow. This lab works through all three, using one running example so the graph only has to be built once.

New ideas in this lab:

- **Path, subpath, length:** a path is a sequence of nodes where each consecutive pair is an edge. Its length is the number of edges, not the number of nodes, so a single node is a path of length 0. A subpath is any contiguous run of nodes taken from within a longer path.
- **Visit vs. tour:** a test path visits a node or an edge if that node or edge appears in it. A test path tours a subpath if the whole subpath appears in it, in order. Every edge is technically a subpath too, so touring is really the general operation and visiting is just the length-0/length-1 special case.
- **Syntactic vs. semantic reachability:** a subpath is syntactically reachable if it exists in the graph on paper. It is semantically reachable if some real test can actually execute

it. Task I from Lab 03 was an example of a node that was syntactically present but not semantically reachable.

- **Prime paths:** a simple path is one where no node repeats, except that the first and last node may be the same, which is how a full loop iteration gets represented. A prime path is a simple path that is not a proper subpath of any other simple path. Prime Path Coverage requires touring every prime path, which is what makes it possible to require a loop run zero times and at least once, without demanding it run every possible number of times.
- **Sidetrips and detours:** a test path tours a subpath q with a sidetrip if every edge of q still appears in the test path in order, except that at one node the test path briefly leaves and comes back to that exact same node before continuing. A detour is more permissive: it only requires every node of q to appear in order, so the test path can leave a node and come back to the next node q needs by any route, not necessarily the direct edge. Both exist because they are sometimes the only way to satisfy an otherwise infeasible test requirement.
- **Data flow coverage:** instead of asking "was this branch taken," these criteria ask "did a value that was stored into a variable actually get read somewhere, before being overwritten." All-Defs Coverage asks for at least one such path per definition, All-Uses Coverage asks for a path to every use of every definition, and All-DU-Paths Coverage asks for every distinct such path, not just one.

Lab Activity

This is an individual activity. Complete each section on your own and type your answers directly into the boxes below.

Section 1: Path and Graph Vocabulary

Prepare a concise document covering the following.

1. What is the difference between a path and a subpath? Why is the length of a path measured in edges rather than nodes, and what does a "path of length 0" actually consist of?

A path is a sequence of nodes connected by edges. A subpath is a continuous part of a path. Path length is the number of edges, so a single node has length 0.

2. Explain the difference between a test path visiting a node or edge and touring a subpath. Is visiting a special case of touring, or are they unrelated ideas?

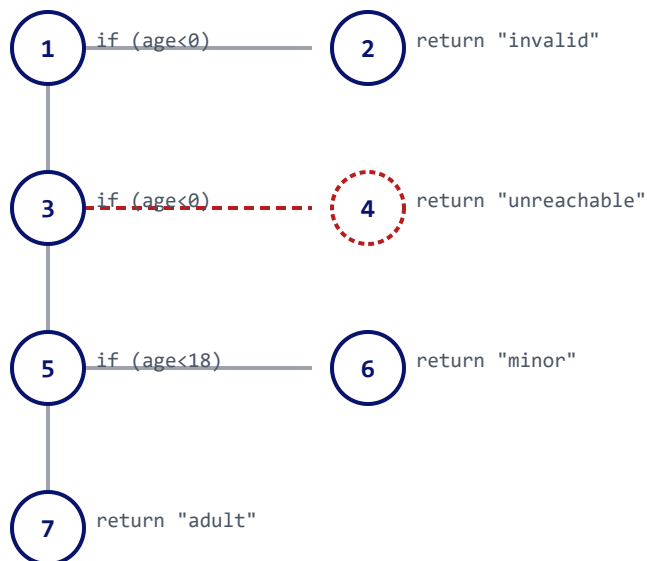
A test path visits a node or edge if it occurs in the path. It tours a subpath if the whole subpath occurs consecutively in the test path.

Visiting is a special case of touring. A node is a length 0 subpath and an edge is a length 1 subpath.

3. What is the difference between syntactic reachability and semantic reachability? Using Lab 03's infeasible-node example as a reference, describe a node or subpath that is syntactically reachable but not semantically reachable.

In the Lab 03 classifyAge example, the "unreachable" return is syntactically reachable because the CFG has a path to it, but it is not semantically reachable because no age value can make both required conditions true.

Interactive: reachability and sidetrips, using Lab 03's own graphs. These are the two graphs you already solved in Lab 03, click around to see the ideas from the definitions above in action. This does not give away anything from this lab's own tasks below.



Click any node to see whether a real call to `classifyAge(age)` can actually reach it, and if so, an `age` value that does.

Click a node to begin. Try node 4 first.

dashed red edge = syntactically present, semantically infeasible

(Interactive graph explorer, open the HTML file in a browser to use it.)

Section 2: Prime Paths and Prime Path Coverage

The Java method below finds the smallest value in an array. As in Lab 03, the code is not pre-numbered, identifying the basic blocks is part of the task. You will reuse this same graph in Section 3, so take care building it here.

```
public static int findMin(int[] values) {  
    int min = values[0];  
  
    for (int i = 1; i < values.length; i++) {  
        if (values[i] < min) {  
            min = values[i];  
        }  
    }  
  
    return min;  
}
```

Interactive: watch findMin run. This only highlights source code as it executes, it does not show you the CFG or any node numbers, that part of Task A is still up to you. Use it to build intuition for the control flow before you formalize it into a graph, or afterwards to check your understanding of a specific path.

(Interactive tracer, open the HTML file in a browser to use it.)

Task A: Write down the graph

Break this method into basic blocks and write down the sets N (nodes), N₀ (initial nodes), N_f (final nodes), and E (edges) for this CFG. Remember the for loop's initialization and its update step each need their own node, the same convention used in Lab 03.

N = {1,2,3,4,5,6,7}

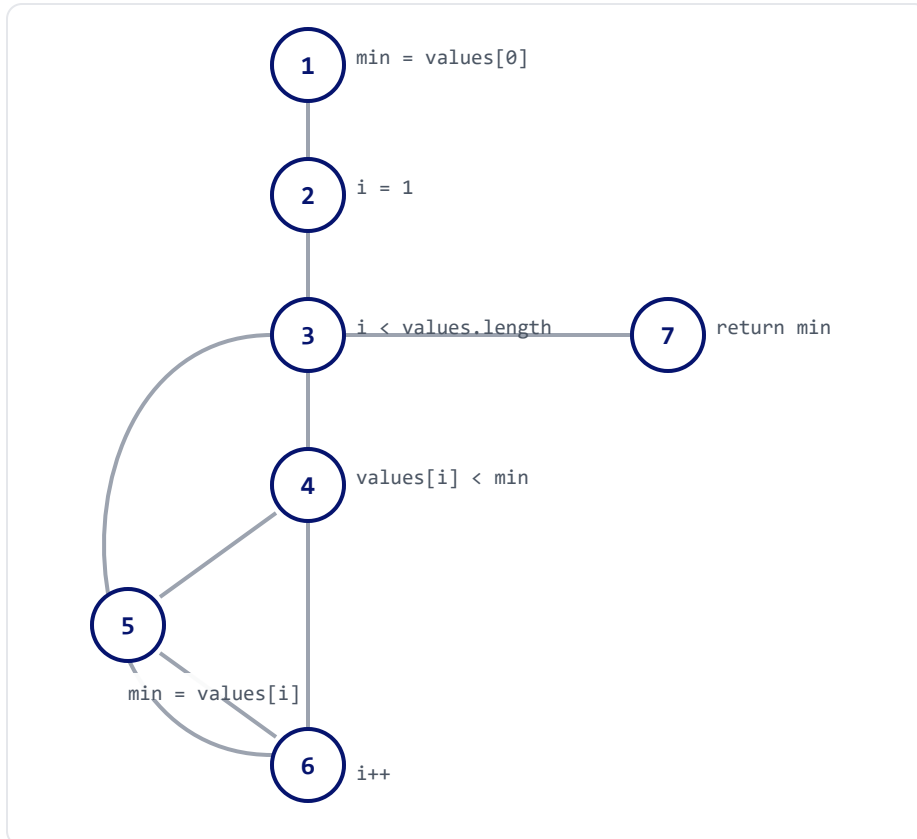
N₀ = {1}

N_f = {7}

E = {(1,2), (2,3), (3,4), (3,7), (4,5), (4,6), (5,6), (6,3)}

Optional but encouraged: draw this CFG in draw.io (or any diagramming tool), export it as a PNG or JPG, and upload it here so you can check your answer visually. You will refer back to this same drawing in Section 3.

Interactive: path, subpath, and prime path explorer for this graph. This is the answer-key CFG for `findMin`, so use it to check your Task A graph, then explore Task B and Task D on it. Click nodes in order to build a path (only edges that really exist in the graph can be followed). The panel classifies your path live.



Path: (click node 1 to start)

length 0

simple?

prime?

Build a path, then type a node sequence above to check whether your path tours it as a contiguous subpath.

(Interactive graph explorer, open the HTML file in a browser to use it.)

Task B: Find the prime paths

Work through this the same way the lecture does: first write down every simple path of length 0 (the nodes themselves) and length 1 (the edges themselves). Then write down the simple paths of length

2, then length 3, and so on, until going further stops producing new simple paths. Remember a simple path cannot repeat a node, except that its first and last node may be the same, that is what lets a path represent one full trip around the loop. Once you have them all, mark which ones are prime, a simple path that is not a proper subpath of any other simple path you found.

Length 0: [(1), (2), (3), (4), (5), (6), (7)]

Length 1: [(1,2), (2,3), (3,4), (3,7), (4,5), (4,6), (5,6), (6,3)]

Length 2: [(1,2,3), (2,3,4), (2,3,7), (3,4,5), (3,4,6), (4,5,6), (4,6,3), (5,6,3), (6,3,4), (6,3,7)]

Length 3: [(1,2,3,4), (1,2,3,7), (2,3,4,5), (2,3,4,6), (3,4,5,6), (3,4,6,3), (4,5,6,3), (4,6,3,4), (4,6,3,7), (5,6,3,4), (5,6,3,7), (6,3,4,5), (6,3,4,6)]

Length 4: [(1,2,3,4,5), (1,2,3,4,6), (2,3,4,5,6), (3,4,5,6,3), (4,5,6,3,4), (4,5,6,3,7), (5,6,3,4,5), (6,3,4,5,6)]

Length 5: [(1,2,3,4,5,6)]

Prime Paths: [(1,2,3,7), (1,2,3,4,6), (1,2,3,4,5,6), (3,4,6,3), (3,4,5,6,3), (4,6,3,4), (4,6,3,7), (4,5,6,3,4), (4,5,6,3,7), (5,6,3,4,5), (6,3,4,6), (6,3,4,5,6)]

Task C: 100% Prime Path Coverage

Write the TR for Prime Path Coverage (every prime path you found in Task B). Then give the smallest set of test cases (array contents) whose test paths, together, tour every prime path. Some prime paths will not be complete test paths on their own, they will need to be toured as part of a longer test path that starts at N0 and ends at Nf.

Test case 1: values = [10] -> prime path toured: [1, 2, 3, 7]

Test case 2: values = [10, 20] -> prime path toured: [1, 2, 3, 4, 6], [4,3,6,7]

Test case 3: values = [10, 5, 3] -> prime path toured: [1,2,3,4,5,6], [3,4,5,6,3], [4,6,3,4], [4,5,6,3,4], [4,5,6,3,7], [5,6,3,4,5], [6,3,4,6], [6,3,4,5,6], [3,4,6,3]

Task D: Sidetrips and detours

Suppose a graph has a prime path that a test path cannot tour directly, perhaps the only way to reach the subpath's first node also forces you through a node the subpath is not supposed to revisit. A sidetrip still preserves every edge of the subpath in order, it just briefly leaves one node and returns to that exact same node before continuing. A detour only needs every node of the subpath to appear in order, so it can reach the next required node by a longer route instead of the direct edge. In your own words, why is a detour able to satisfy some test requirements that a sidetrip cannot? Then check your Task B answer: does touring any of this method's prime paths actually require a sidetrip or a detour, or can every one of them be toured directly by some real test path? Explain why.

A sidetrip must leave a node and return to that same node, while a detour only requires the nodes of the required subpath to occur in order. Therefore, a detour can use a different route to reach the next required node.

This graph doesn't need either. Every prime path in this graph can be toured directly by a valid test path, so no sidetrips or detours are needed.

Section 3: Data Flow Coverage

This section reuses the exact same `findMin` graph from Section 2. Data flow criteria do not look at branches directly, they track two things for each variable: where it is **defined** (a value is stored into it) and where it is **used** (its value is read). A definition and a use only count as a DU pair if there is a def-clear path between them, a path that does not store a new value into the variable anywhere in between.

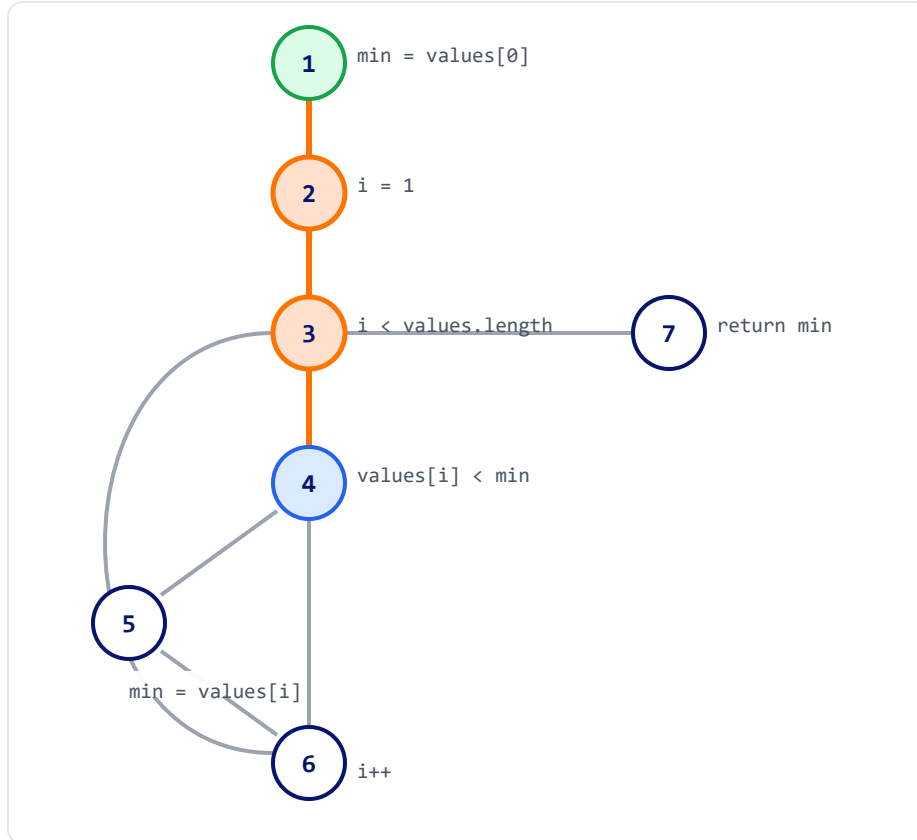
What counts as a def or a use: a def happens when a variable is assigned, is a method parameter (an implicit def where the method begins), or receives a value from user input. A use happens when a variable appears on the right-hand side of an assignment, appears in a conditional test, is passed as an argument, or is read in a return statement.

Task E: Fill in the def and use sets

Focus on the two variables `min` and `i`. For each node in your Task A graph that defines or uses either variable, write down `def(node)` and `use(node)`. (You do not need to track values, it is only ever read, never reassigned.)

Node 1: `def(1) = {min}, use(1) = {}`
Node 2: `def(2) = {i}, use(2) = {}`
Node 3: `def(3) = {}, use(3) = {i}`
Node 4: `def(4) = {}, use(4) = {i, min}`
Node 5: `def(5) = {min}, use(5) = {i}`
Node 6: `def(6) = {i}, use(6) = {i}`
Node 7: `def(7) = {}, use(7) = {min}`

Interactive: defs, uses, and DU-paths on this same graph. Use this to check your Task E answer, then to see the actual def-clear path for each DU pair before you write out Tasks F through I.



def of selected pair use of selected pair same node, both connecting du-path

DU pair (1,4): def shown in green, use shown in blue, connecting du-path in orange. Exactly 1 du-path: [1, 2, 3, 4]

(Interactive graph explorer, open the HTML file in a browser to use it.)

Task F: DU pairs and DU paths for min

`min` is defined in more than one place. List every DU pair (a def location and a use location connected by a def-clear path) for `min`, and for each pair, write down the actual def-clear path (the du-path) that connects them.

(1,4) - [1,2,3,4], (1,7) - [1,2,3,7], (5,4) - [5,6,3,4], (5,7) - [5,6,3,7]

Task G: All-Defs Coverage

Write the TR for All-Defs Coverage on `min`: for each definition of `min`, at least one du-path to some use must be toured. Give a set of test cases that satisfies it.

TR: [(1,2,3,4), (5,6,3,4)]

Test cases: values = [10, 5, 20]

Task H: All-Uses Coverage

Write the TR for All-Uses Coverage on `min` : for each definition, a du-path to every use must be toured, not just one. Give a set of test cases that satisfies it, and explain in one or two sentences why this needed more test requirements than Task G, given that `min` has more than one use reachable from at least one of its definitions.

TR: [(1,2,3,4), (1,2,3,7), (5,6,3,4), (5,6,3,7)]

Test cases: values = [10], values = [10,5], values = [10,5,20]

Task I: All-DU-Paths Coverage

All-DU-Paths Coverage requires touring every distinct du-path for a DU pair, not just one. Look back at your Task F answer: does any DU pair for `min` in this particular graph actually have more than one du-path connecting it? If not, does that mean All-Uses Coverage and All-DU-Paths Coverage require exactly the same test cases here? In general, what would have to be true about the graph, structurally, for a DU pair to have more than one du-path? Try switching the explorer above to `i` and stepping through its DU pairs, you will find one there.

Each DU pair for `min` has only one def-clear path, so All-Uses and All-DU-Paths have the same requirements here.

Multiple alternative def-clear routes between the same definition and use, usually caused by branches or loops in the CFG.

Task J: Subsumption and Complete Path Coverage

Edge-Pair Coverage is generally stronger than Edge Coverage, and Prime Path Coverage is generally stronger than both, in the sense that satisfying the stronger criterion guarantees satisfying the weaker one. Using what you found in Lab 03's Task H (the self-loop example), is that guarantee ever broken? Which pair of criteria did that involve, and why?

A self-loop can break the guarantee because Edge-Pair Coverage can require paths containing repeated nodes, while prime paths must be simple.

Complete Path Coverage (CPC) requires touring every possible path in a graph, not just every prime path. For any graph that contains a loop, explain why CPC is essentially always infeasible, and why Prime Path Coverage was specifically designed to give most of the same confidence without that problem.

A loop can execute any number of times, creating infinitely many possible paths. Prime Path Coverage avoids this by covering finite maximal simple paths and loop paths instead.

Discussion

Node Coverage, Edge Coverage, and Edge-Pair Coverage all look only at control flow. All-Defs, All-Uses, and All-DU-Paths Coverage look only at variables. Give one example of a fault that a data flow criterion would catch but that Edge-Pair Coverage could miss entirely, even at 100%.

A variable could be assigned an incorrect value while the control flow remains unchanged. For example, changing `total = price - 10` to `total = price - 100` would not affect Edge-Pair Coverage, but data-flow testing could detect the incorrect definition reaching the use.

Across Labs 02, 03, and this lab, you have now met Node, Edge, Edge-Pair, and Prime Path Coverage, plus All-Defs, All-Uses, and All-DU-Paths Coverage. If you only had time to apply one criterion to a piece of code with a loop and several variables, which would you pick, and what would you accept giving up by not using the others?

would choose Prime Path Coverage because it provides strong control-flow coverage and specifically handles loops while remaining finite. The trade-off is that it does not specifically check definition-use relationships, so some data-flow faults could be missed.

